



分类算法与KNN、决策树学习笔记

基于《白话机器学习算法》第7、9章

MACHINE LEARNING ALGORITHMS REVIEW

目录



1. 分类算法基础



2. KNN算法详解



3. 决策树算法详解



4. KNN与决策树对比



5. 总结与附录



一、分类算法的基本逻辑与概念

BASIC LOGIC AND CONCEPTS OF CLASSIFICATION ALGORITHMS



分类算法的基本逻辑

🕒 核心定位

分类算法属于**监督学习**的一种，是机器学习中最基础且应用最广泛的任务之一。

💡 核心逻辑

利用**已知类别标签**的训练数据，学习特征到类别的映射函数，进而对**未知数据**进行类别预测。

🔄 基本流程

- 准备数据：收集并整理带有明确类别标签的历史数据。
- 训练模型：划分训练集与测试集，训练分类器学习规律。
- 评估预测：验证模型性能（如准确率），并应用于新数据。

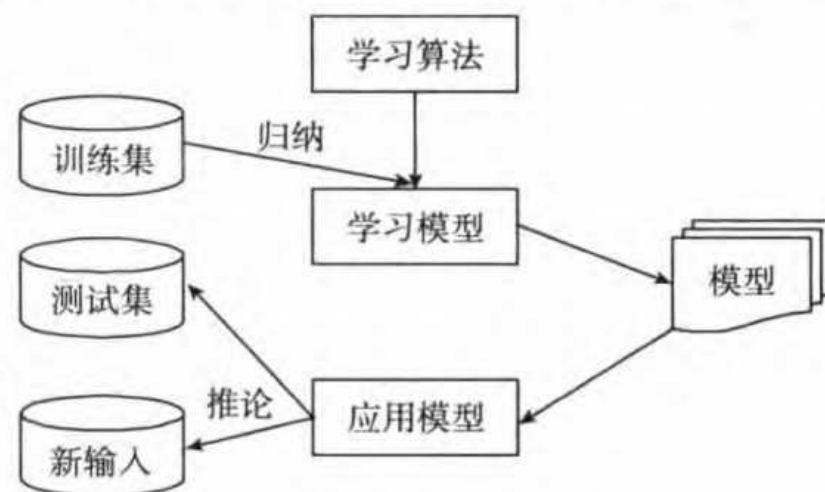


图 8-1 分类原理示意图

分类算法基本概念



核心概念	概念解释
分类器 (Classifier)	能够解释原始数据、预测新数据类别的模型（如KNN、决策树）。
训练集 (Training Set)	用于训练模型的原始数据，包含特征和已知的类标签。
类标签 (Class Label)	原始数据中已经为每个记录标记好的类别（如“健康/疾病”）。
测试集 (Test Set)	从原始数据中划分出的一部分，用于评估模型的泛化能力。
开放集 (Open Set)	使用训练好的分类器进行分类工作的全新数据（无标签）。
过拟合 (Overfitting)	模型过度学习训练数据中的噪声，导致对新数据预测能力差。
欠拟合 (Underfitting)	模型过于简单，未能捕捉数据中的基本模式。



CHAPTER 02

二、KNN算法的思想与步骤

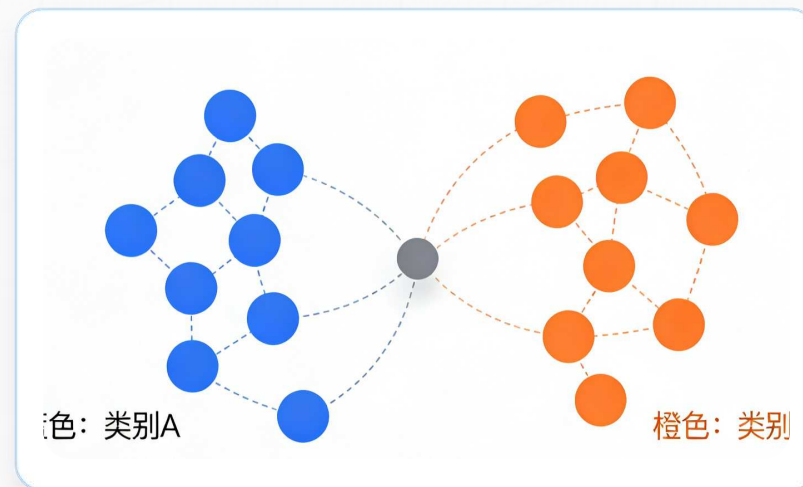
KNN核心思想：物以类聚

核心思想：近朱者赤

一个未知样本的类别，由其周围最近的 K 个已知样本的类别“少数服从多数”投票决定。

通俗解释：看朋友圈

就像判断一个人的身份，可以观察他身边最亲近的几个朋友是做什么的，通过多数人的特征来推断他的特征。



图示：未知样本（灰色）由周围邻居投票决定类别



01 度量距离

计算待分类样本到每个训练样本之间的距离（常用欧氏距离）。



02 选择邻居

选取距离最近的K个训练样本（K值需预先设定）。

KNN算法步骤

KNN 算法具体步骤如下：

- 步骤 1：初始化距离为最大值。
- 步骤 2：计算未知样本和每个训练样本的距离 $dist$ 。
- 步骤 3：得到目前 K 个最近邻样本中的最大距离 $maxdist$ 。
- 步骤 4：如果 $dist$ 小于 $maxdist$ ，则将该训练样本作为 K-最近邻样本。
- 步骤 5：重复步骤 2~步骤 4，直到未知样本和所有训练样本的距离都算完。
- 步骤 6：统计 K 个最近邻样本中每个类别出现的次数。
- 步骤 7：选择出现频率最大的类别作为未知样本的类别



03 统计类别

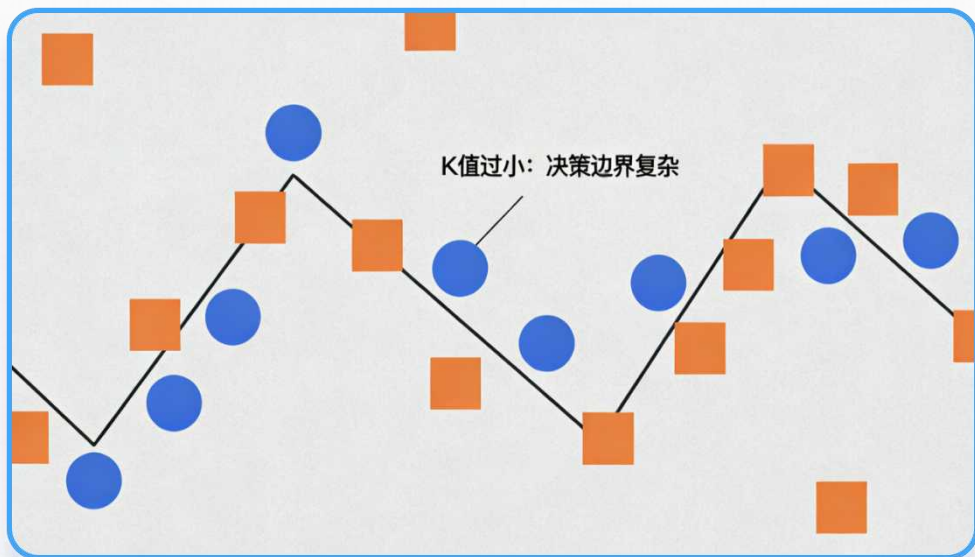
统计这K个邻居中每个类别出现的次数。



04 投票决定

将待分类样本归入出现次数最多的类别。

K值的选择与注意事项



图示：不同K值对决策边界的影响



K值过小 (过拟合)

模型过于复杂，容易受噪声干扰，决策边界不平滑。



K值过大 (欠拟合)

纳入过多远距离样本，忽略局部特征，导致分类错误。



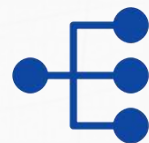
合适K值 (最佳)

通过交叉验证(Cross Validation)选择，使误差最小化。



加权投票改进

采用距离加权，邻近样本权重更高，提升分类准确性。



三、决策树算法详解

DECISION TREE ALGORITHM

决策树基本结构

根节点 (Root Node)

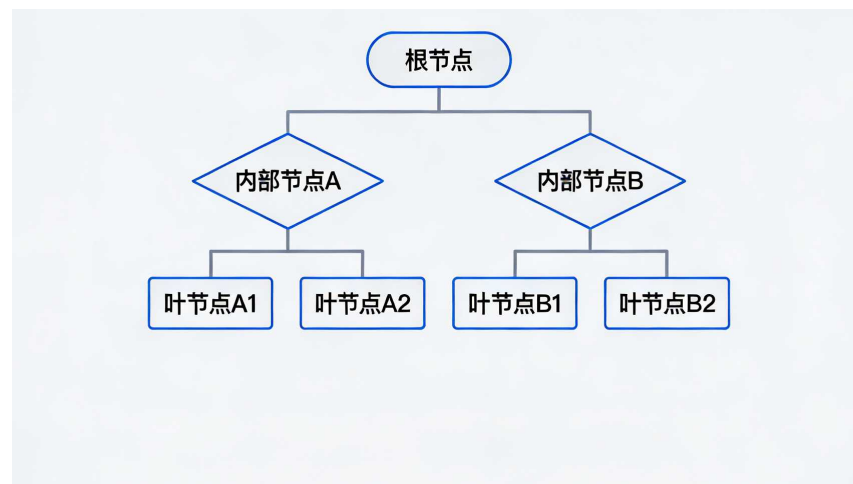
位于树的最顶端，代表第一个需要判断的核心问题。

内部节点 (Internal Node)

树中间的分支点，通常是二元选择题（是/否），引导向下分类。

叶节点 (Leaf Node)

树的末端，代表最终的预测结果、类别或概率值。



 **结构特点：**每个分支仅有两个答案（是/否），层层递进形成清晰的树状逻辑结构。

决策树生成方法：递归拆分



选择最优拆分

选择一个能最大化子集同质性的二元问题来拆分数据，使子集尽可能“纯”。



递归拆分过程

对每个拆分出的子节点重复执行“选择最优拆分”的操作，层层深入。



算法终止条件

- 叶节点样本全属于同一类别
- 叶节点样本数小于设定阈值
- 进一步拆分无法提高同质性



决策树递归生成流程示意

决策树的优缺点

✓ 核心优势 (Pros)

易于理解与可视化

决策逻辑直观透明，生成的树状图便于解释和沟通。

对异常值鲁棒性强

基于阈值分割，不易受极端值干扰，无需复杂的数据预处理。

特征影响自动筛选

不显著的变量对最终结果影响较小，具有一定的特征选择能力。

💡 局限性 (Cons)

模型结构不稳定

数据的微小变动可能导致生成完全不同的树结构，泛化能力较弱。

易陷入局部最优

采用贪心算法，只保证局部最优，无法确保得到全局最优解。

容易发生过拟合

当树的深度过大时，模型会过度学习训练数据的细节，降低泛化能力。



总结：决策树是优秀的基准模型和解释性工具，但在实际应用中，通常通过集成学习（如随机森林、GBDT）来克服其不稳定性和过拟合问题。

决策树的改进方法

- **随机森林**：随机选择不同的二元选择题，生成多棵决策树，然后综合这些决策树的预测结果。第 10 章将详解这种方法。



随机森林 (Random Forest)

集成多棵随机生成的决策树，通过投票来决定最终结果。可以有效提高模型的稳定性和准确性。



梯度提升 (Gradient Boosting)

逐步构建一系列决策树，每一棵新树都试图修正前一棵树的错误。也能显著提升模型性能。

- **梯度提升**：有策略地选择二元选择题，以逐步提高决策树的预测准确度；然后将所有预测结果的加权平均数作为最终结果。



代价与权衡

这些改进方法虽然效果更好，但模型会变得更加复杂，失去了决策树原有的可解释性，变成了“黑盒”模型。



四、KNN与决策树对比

算法特性解析与应用场景深度剖析

KNN与决策树对比表

特性维度	KNN (K近邻)	决策树 (Decision Tree)
训练阶段	无训练（惰性学习），仅存储所有数据，预测时计算	有显式训练过程，基于训练集构建树结构模型
预测速度	较慢（需计算与所有样本的距离并排序）	较快（沿树路径向下遍历即可得出结果）
可解释性	较差（基于邻居投票机制，逻辑不直观）	优秀（树结构可视化，规则清晰易懂）
对异常值	敏感（异常点会干扰局部邻居分布）	不敏感（基于特征阈值拆分，受单点影响小）
高维数据表现	表现差（受维度灾难影响，距离计算失效）	相对较好（能自动选择重要特征进行拆分）



五、总结与附录

SUMMARY AND APPENDIX



关键术语速查

术语	解释
k最近邻	根据待分类点周围距离最近的k个数据点的类型进行分类。
决策树	通过一系列二元选择题来拆分数据样本以获得同质组的监督学习技术。
递归拆分	反复拆分数据样本以得到同质组的过程。
过拟合	模型对数据中的随机波动过于敏感，泛化能力差。
交叉验证	将数据集划分为若干组，反复训练和测试，取平均准确度。
混淆矩阵	展示真实类别与预测类别对应关系的表格，用于计算假正例、假负例等。



总结



分类算法核心

分类算法是监督学习的核心，目标是学习从特征到类别的映射关系。



KNN 算法特性

属于惰性学习，思想简单直观，但预测速度较慢，对高维数据表现不友好。



决策树算法特性

属于积极学习，模型可解释性强，预测速度快，但模型复杂度高时容易过拟合。



算法选择策略

需根据数据特点、性能要求和可解释性需求综合考虑，选择最适合的模型。



感谢观看

THANKS FOR WATCHING

